TripStack Capstone – UI Automation Progress Document

Name:Karthikeyan S

Date:17-07-2026

Project Name: TripStack Booking Automation Framework

Application:

- Web UI: https://tripstack.doomple.com

Automation Tool: Playwright

Language: TypeScript

Framework Architecture: Layered Automation Framework

Objective

Develop a scalable and maintainable automation framework capable of validating the complete TripStack booking lifecycle.

The final capstone will include:

- UI Automation
- API Automation
- Database Validation
- Security Testing
- Performance Testing
- CI/CD Integration

Currently, the UI automation portion has been completed.

Technology Stack

Component	Technology
Language	TypeScript
UI Automation	Playwright
Design Pattern	Page Object Model

Test Runner	Playwright Test
Build Tool	Node.js / npm
Version Control	Git
Repository	GitHub

Framework Architecture

The framework follows a layered architecture for better maintainability and scalability.

Tests

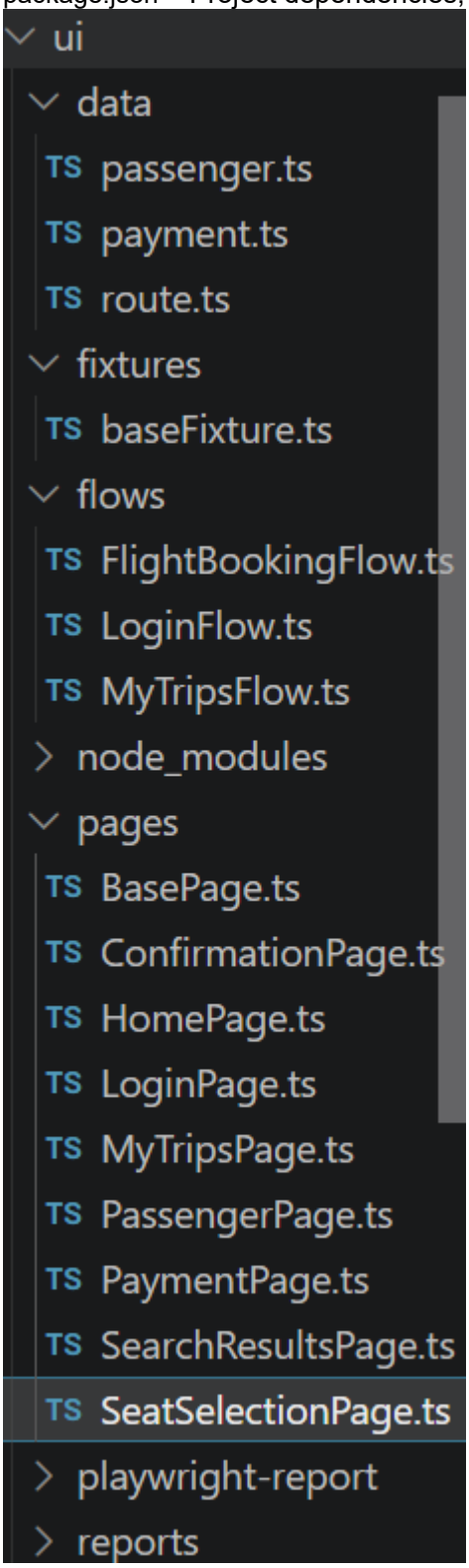
Flows

Page

BasePage

Playwright

- **Pages**
  - SearchResultsPage.ts
  - SeatSelectionPage.ts
  - PassengerPage.ts
  - PaymentPage.ts
  - ConfirmationPage.ts
  - MyTripsPage.ts
- **Flows**
  - LoginFlow.ts – Handles user login flow.
  - FlightBookingFlow.ts – Handles end-to-end flight booking flow.
  - MyTripsFlow.ts – Handles trip management and verification flow.
- **Fixtures**
  - baseFixture.ts – Common Playwright fixtures and test setup.
- **Data**
  - route.ts – Route and flight search test data.
  - passenger.ts – Passenger details test data.
  - payment.ts – Payment-related test data.
- **Utilities**
  - DateUtil.ts – Date manipulation and formatting utilities.

    - RandomUtil.ts – Random data generation utilities.
- **Tests**
    - booking.spec.ts – Main flight booking test scenarios.
- **Configuration Files**
    - .env – Environment variables and secrets.
    - playwright.config.ts – Playwright framework configuration.
    - package.json – Project dependencies, scripts, and metadata.
    - 

- ---

Framework Layers

Test Layer

The test layer contains only business scenarios.

Responsibilities:

- Call flows
- Execute end-to-end scenarios
- Maintain readability
- No locators

Flow Layer

The flow layer combines multiple page actions into reusable business processes.

LoginFlow

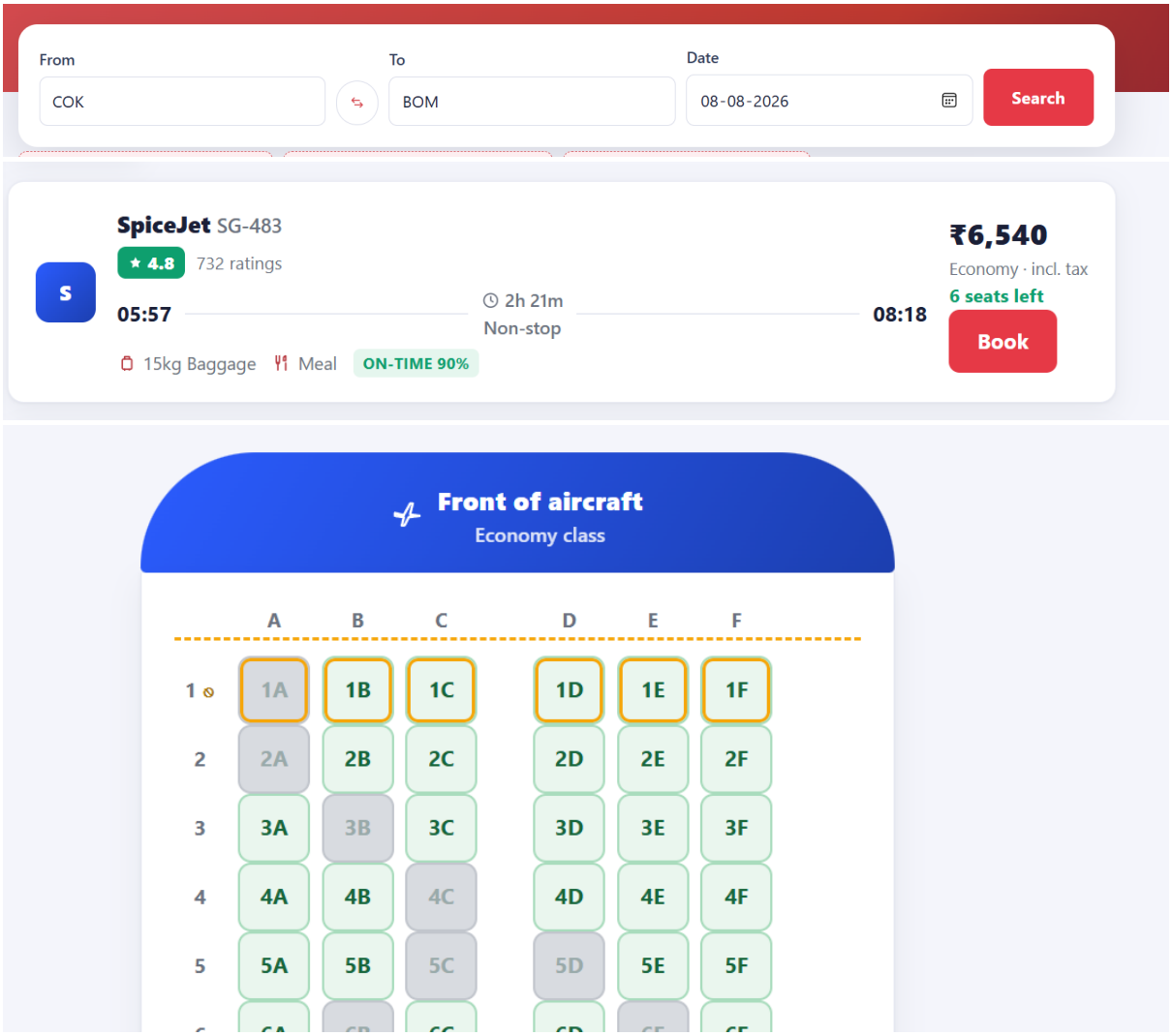
Responsibilities

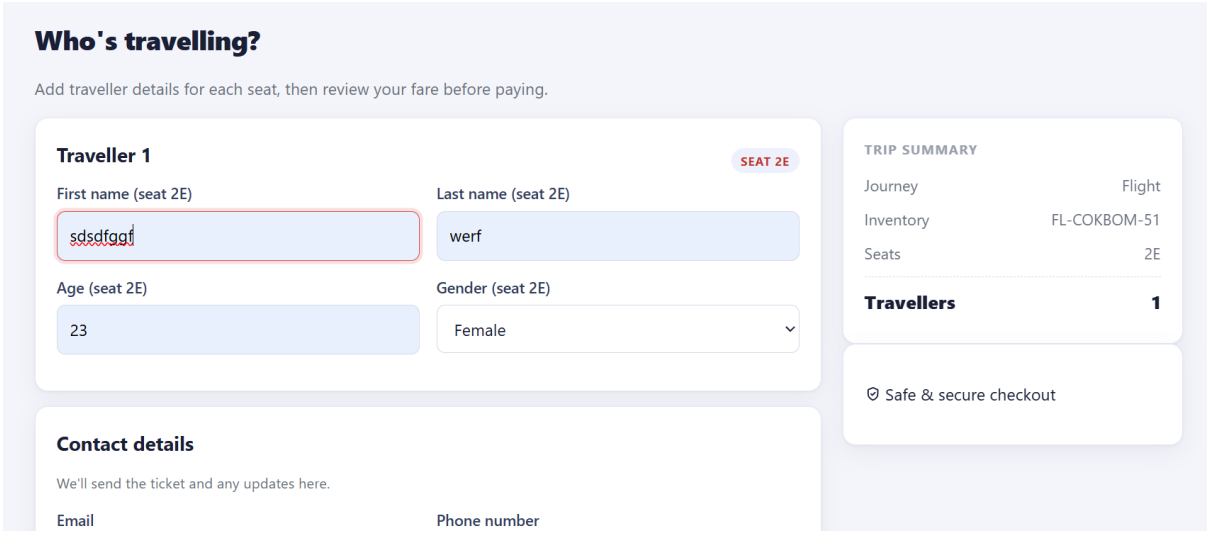
- Login into application

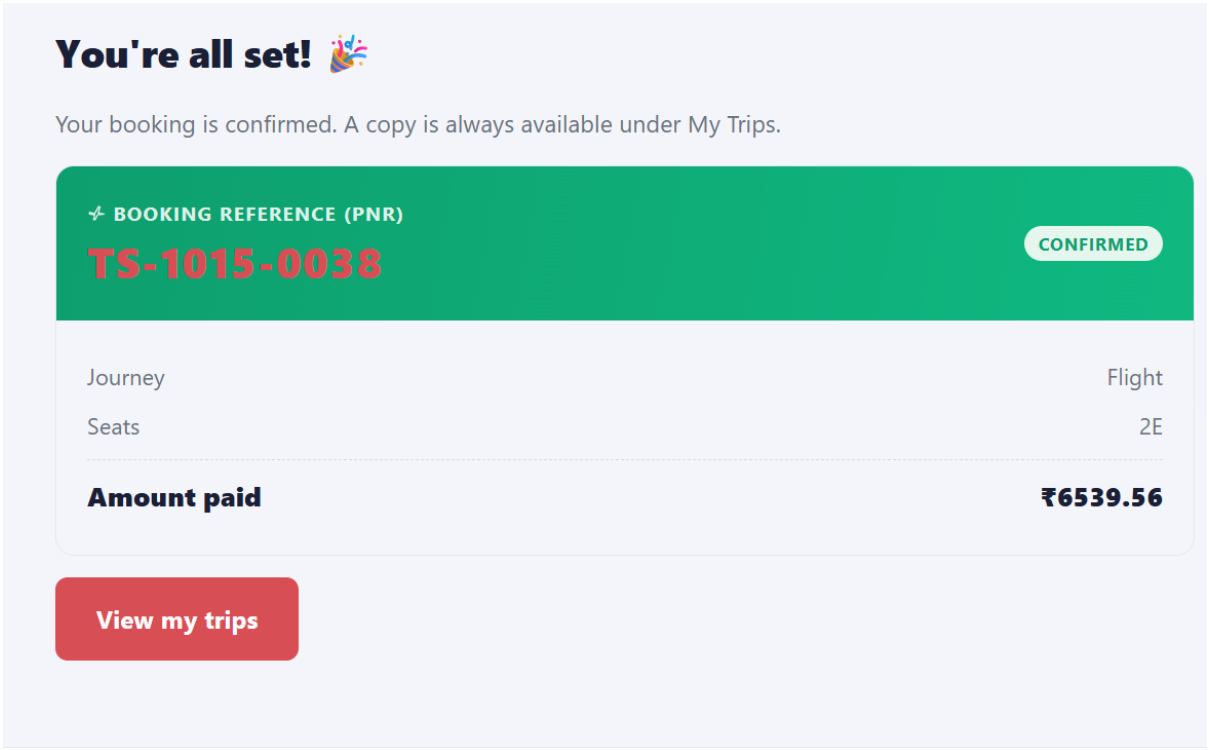
FlightBookingFlow

Responsibilities

- Search Flight
- Select Flight
- Select Seat
- Passenger Details
- Payment
- Booking Confirmation

- 
From
COK
To
BOM
Date
08-08-2026
Search
SpiceJet SG-483
4.8
732 ratings
05:57
2h 21m
Non-stop
08:18
15kg Baggage
Meal
ON-TIME 90%
₹6,540
Economy · incl. tax
6 seats left
Book
Front of aircraft
Economy class
A B C D E F
1 1A 1B 1C 1D 1E 1F
2 2A 2B 2C 2D 2E 2F
3 3A 3B 3C 3D 3E 3F
4 4A 4B 4C 4D 4E 4F
5 5A 5B 5C 5D 5E 5F





MyTripsFlow

Responsibilities

- Open My Trips
- Verify Booking
- Cancel Booking
- Verify Refund Status

Page Layer

Each page contains only:

- Locators
- Page actions

Business logic is never written inside page classes.

BasePage

A common parent class created for all pages.

Purpose

- Eliminate duplicate Playwright code
- Increase maintainability
- Improve readability

Reusable methods include:

- navigate()
- click()
- fill()
- selectOption()
- getText()
- waitForElement()
- verifyUrl()
- takeScreenshot()

All page classes extend BasePage.

Test Data Management

Static test data is separated from business logic.

Route Data

Contains:

- From City
- To City

- Travel Days

Passenger Data

Contains:

- First Name
- Last Name
- Age
- Gender

Payment Data

Contains:

- Card Holder
- Card Number
- Expiry
- CVV

Environment Variables

Sensitive information is stored in the .env file.

Contains:

- BASE_URL
- EMAIL
- PASSWORD
- PHONE
- EMPLOYEE_ID

No credentials are hardcoded.

Stable Locator Strategy

Locator priority followed:

1. id

2. data-id
3. name
4. Role-based locators
5. CSS
6. XPath (only where necessary)

This improves locator stability and reduces maintenance.

Dynamic Passenger Handling

Seat numbers are dynamic.

Instead of:

- #name-1A

Dynamic locators are used:

- input[name^="firstName_"]
- input[name^="lastName_"]
- input[name^="passengerAge_"]
- select[name^="passengerGender_"]

This allows the framework to work regardless of the selected seat.

Dynamic Seat Selection

Instead of selecting a fixed seat such as:

- 1A
- 16F

The framework:

- Counts all available seats.
- Randomly selects one available seat.
- Continues the booking.

Benefits:

- Avoids seat conflicts.
- Prevents repeated failures due to previously booked seats.

- Improves test reliability.

Travel Date Handling

Travel dates are generated dynamically.

Current implementation:

Today's Date + 22 Days

This avoids failures caused by expired dates.

PNR Validation

After successful booking:

- PNR is captured.
- Booking status is verified.
- PNR format is validated.

Validation Pattern:

TS-XXXX-XXXX

Example:

TS-1015-0003

UI Automation Flow Completed

The following end-to-end booking lifecycle has been automated:

Step 1

Launch Application

Step 2

Login

- Email
- Password

Step 3

Search Flight

- From: COK
- To: BOM
- Date: Today +22

Step 4

Verify Search Results

- Verify flights are displayed.

Step 5

Book Flight

- Select first available flight.

Step 6

Seat Selection

- Verify seat map.
- Select random available seat.

Step 7

Passenger Details

Enter:

- First Name
- Last Name
- Age
- Gender

Step 8

Contact Details

Enter:

- Email
- Phone Number

Step 9

Payment

Enter:

- Card Holder
- Card Number
- Expiry
- CVV

Step 10

Booking Confirmation

Verify:

- Booking Successful
- Booking Status
- Capture PNR

Step 11

My Trips

Navigate to:

My Trips

Verify:

Booking exists.

Step 12

Cancel Booking

Cancel the booking.

Step 13

Refund Verification

Verify booking status changes to:

Refunded

Current Features Implemented

- Login Automation
- Flight Search
- Flight Booking
- Dynamic Seat Selection
- Dynamic Passenger Locators
- Contact Details
- Payment
- Booking Confirmation
- PNR Validation
- My Trips Verification
- Booking Cancellation
- Refund Validation
- BasePage Implementation
- Layered Framework
- Flow Layer
- Fixtures
- Environment Variables
- Stable Locator Strategy

Multi-City Flight Booking

According to the assigned capstone scenario, the framework will be extended to support Multi-City flight booking.

Flow

1. Login.
2. Search first flight:
    - COK → BOM
3. Complete the first booking.
4. Navigate to **My Flights / My Trips**.
5. Add the next flight.
6. Search second flight:
    - BOM → DEL (or the destination specified in the assignment)
7. Select the flight.
8. Select a seat.
9. Complete passenger and payment steps.
10. Verify the updated itinerary.
11. Cancel the booking and verify the refund status if required.

Framework Impact

The existing framework is designed to support this enhancement with minimal changes.

The following components are expected to be extended:

- HomePage
- FlightBookingFlow
- Route Data

The remaining components can be reused without modification:

- SearchResultsPage
- SeatSelectionPage
- PassengerPage
- PaymentPage
- ConfirmationPage
- MyTripsPage

Conclusion:

End to end flow journey and multiflight has been implemented and tested and verified successfully.